

Module 2 — Tool Calling

The core pattern for AI agents

🎯 Learning goals

- Understand the **message format** that drives agent conversations.
- Build a **tool-calling loop** from scratch.
- Implement a **tool registry** with validation and routing.
- Add **safety rails**: allowlists, rate limits, audit logs.

Message roles

- **system** — sets behaviour, available tools, constraints.
- **user** — the human (or upstream agent) input.
- **assistant** — the LLM response: text and/or tool calls.
- **tool** — result of a tool execution, linked by call_id.

Passing tools to the model

- Tools are passed as a **`tools` parameter** alongside the messages in every API call.
- Each tool is a JSON object describing its **name**, **description**, and **parameter schema** — the model never sees your code, only this specification.
- The model reads the descriptions to decide **which tool to call** and what arguments to pass — good descriptions are critical.
- Tools are **not persistent** — you send the full list with every request, just like the message history.
- The response's **`output`** contains **`function_call` items** (tool name, JSON arguments, `call_id`) when the model wants a tool, or a text message if no tool is needed.

Conversation state

```
messages = [
    {"role": "system", "content": "You are a helpful assistant..."},
    {"role": "user", "content": "How many engineers are in the London office?"},
    {"type": "function_call", "call_id": "c1",
     "name": "lookup_employees", "arguments": '{"office": "London"}'},
    {"type": "function_call_output", "call_id": "c1",
     "output": '[{"name": "Alice"}, ...]'},
    {"role": "assistant",
     "content": "3 engineers found: Alice, Bob, Carol."},
]
```

- The LLM decides whether to call a tool or answer directly
- Tool results go back as function_call_output items, matched by call_id



Demo — Tool-call message flow

Switch to terminal: python demo/demo.py → Part 2



The tool-calling loop

From LLM decision to tool execution and back

The core loop

```
def run_tool_loop(client, tools, handlers, user_input, max_steps=10):
    messages = [system_msg, {"role": "user", "content": user_input}]

    for _ in range(max_steps):
        response = client.responses.create(model=MODEL, input=messages, tools=tools)
        calls = [i for i in response.output if i.type == "function_call"]

        if calls:
            messages += response.output          # keep the model's calls
            for tc in calls:
                result = handlers[tc.name](**json.loads(tc.arguments))
                messages.append({"type": "function_call_output",
                                 "call_id": tc.call_id, "output": result})
        elif response.output_text:
            return response.output_text           # final answer

    return None  # exhausted steps
```

- → max_steps prevents infinite loops
- → Each tool result is appended so the LLM sees the full context

Tool registry pattern

- Tools declare a **JSON schema**: name, description, parameters.
- Registry **validates** arguments before calling the handler.
- Registry **routes** by name and **catches** handler exceptions.
- OpenAI-compatible format: `{"type": "function", ...}`.

Registering a tool

```
registry = ToolRegistry()

@registry.register(
    name="check_inventory",
    description="Check current inventory for a product",
    parameters={
        "type": "object",
        "properties": {
            "product": {"type": "string"}
        },
        "required": ["product"],
    },
)
def check_inventory(product: str) -> dict:
    return {"product": product, "stock": 42}
```

- → Decorator pattern: schema lives next to the handler
- → JSON schema doubles as documentation and validation spec



Demo — Tool registry

Switch to terminal: python demo/demo.py → Part 3



Safety rails

Allowlists, rate limits, and audit trails

Why safety rails?

- LLMs are **unpredictable** — they can hallucinate tool names, loop indefinitely, or leak sensitive data.
- An unguarded agent is fine for demos. **Production requires boundaries.**
- Safety rails sit between the model's decision and the actual tool execution — a checkpoint layer.
- The model still sees denied calls as error messages, so it can **adapt and explain** rather than crash.

Allowlists

- **What:** A set of permitted tool names. Any call to a tool not in the set is rejected before execution.
- **Why:** The model might hallucinate a tool name or try to call a destructive tool it shouldn't have access to.
- **Without it:** The agent can call *anything* — including tools you never intended to expose.
- **Example:** Permit `scan_planet` and `check_habitability`, block `log_discovery` — the agent can read but not write.

Rate limiting

- **What:** Cap how many tool calls can happen within a sliding time window.
- **Why:** A confused or looping agent can fire hundreds of calls in seconds, burning through API budget and hammering downstream services.
- **Without it:** One bad prompt can trigger an infinite tool-call loop with no brake.
- **Pattern:** Track timestamps of recent calls, prune those outside the window, reject if the count exceeds the limit.

Redaction

- **What:** Strip sensitive data — API keys, PII, credentials, internal IDs — from tool results before they reach logs or the model.
- **Why:** Tool results often contain raw data from internal systems. That data can end up in audit logs, conversation history, or even shown to the user.
- **Without it:** Secrets silently leak into stored conversations, log aggregators, or downstream models.
- **Pattern:** Regex or pattern-based replacement applied to every tool result before logging.

Audit logging

- **What:** Record every tool call — allowed or blocked — with timestamp, tool name, arguments, result, and outcome.
- **Why:** When an agent misbehaves, the audit log is the only way to reconstruct what happened and why.
- **Without it:** No trail to debug incidents, measure usage, or prove compliance to stakeholders.
- **Each entry records:** was the call allowed? What arguments were passed? What came back? When did it happen?

Blocked tool UX

- **What:** When a tool is denied, return a structured error message as the tool result — don't crash or silently skip.
- **Why:** The model needs to *see* the denial so it can explain the situation to the user or try an alternative approach.
- **Without it:** The conversation breaks — the model expects a tool result and gets nothing, or the program crashes.
- **Example:** Return `{"error": "Tool not permitted: log_discovery"}` — the model reads this and says *"I can't log discoveries with my current permissions."*



Demo — Guarded agent

Switch to terminal: python demo/demo.py → Part 4



Putting it all together

Summary and exercises

Summary

Messages drive the loop

system, user, assistant, and tool results are the whole conversation.

Cap the loop

max_steps prevents runaway agents and surprise bills.

Validate before you execute

Check allowlist and schema before calling any tool.

Log every tool call

An audit trail is your best debugging tool when agents misbehave.



Exercises

Time to build

Getting started

- Activate your venv: **`source .venv/bin/activate`**.
- Each exercise has a **`start.py`** (your work) and **`test_start.py`** (pytest).
- Run tests with: **`pytest module-02-tool-calling/exercises/01-tool-calling-agent/`**.
- Solutions are in **`solution.py`** — try the exercise first!

Exercises

01 — Tool-calling agent: the minimal LLM → tool → result cycle

02 — Tool registry: schema, validation, routing

03 — Guarded agent: allowlists, rate limits, and audit logs



Module 2 — Complete

Next: MCP and tool integration